Plateforme de financement participatif à gouvernance par jeton

Crowdfund DAO

Rapport technique corrigé
Module Blockchain - ENSA Beni Mellal
Version vérifiée le 4 novembre 2026

Objet et résultat

Le système collecte des ETH dans des campagnes, crée des jetons CFD proportionnels aux contributions et soumet les paiements à une décision de gouvernance. Quatre contrats séparent le jeton, le séquestre, le vote et la trésorerie multisignature. Cette version corrige deux écarts majeurs : la conservation de voix après remboursement et le retrait d'une campagne réussie sans autorisation de la DAO.

La version corrigée passe 39 tests automatisés. Le front-end compile, les métadonnées du CID fourni sont chargées et une proposition de démonstration a été exécutée automatiquement sur Hardhat local. Les quatre contrats sont de plus déployés sur le testnet Sepolia (chaîne 11155111), bloc 12085541, et leur code source y est publié sur Sourcify en correspondance exacte. Le rapport ne présente pas les signatures réelles dans MetaMask comme réalisées.

Lecture du rapport	Pages
Exigences et périmètre	2
Architecture et rôles	3
Jeton et remboursement	4
Cycle de vie et retraits	5
Gouvernance et automatisation	6
Sécurité	7
Tests et audit statique	8
Déploiement et capture	9
Recette, bonus et limites	10

Copie de référence : C:/Users/thund/Downloads/block chaine. La copie OneDrive contient une version antérieure et ne doit pas être confondue avec la version exécutée.

Exigences et périmètre

Le cahier des charges impose une dApp multi-contrats en Solidity, un jeton ERC-20 OpenZeppelin, une collecte en ETH, des votes pondérés avec snapshot et des mécanismes de sécurité. La démonstration sur blockchain locale Hardhat est explicitement autorisée. Les obligations sont mises en correspondance avec le code ci-dessous.

Exigence	Implémentation corrigée
M1 - Jeton CFD	ERC20 et ERC20Votes ; 100 CFD par ETH ; mint automatique ; poids de vote egal au solde.
M2 - Campagnes	Création, contribution, statuts, remboursement et paiement au bénéficiaire.
M2 - Validation du retrait	Toute sortie au bénéficiaire exige une approbation DAO exécutée.
M3 - Vote	Actions de déblocage anticipé, gel et validation du retrait final ; quorum 50 % ; majorité absolue de l'offre snapshot.
M3 - Exécution	Délai en blocs et exécuteur automatique dédié à Hardhat local.
M4 - Interface	React, Ethers.js, connexion MetaMask, liste, collecte et tableau de vote ; actualisation toutes les 4 s.
ABI et IPFS	ABI exportées depuis la compilation ; hash seul on-chain et métadonnées sur IPFS.
Tests et fuzzing	39 tests ; scénario de 18 contributions pseudo-aléatoires à graine fixe.
Sécurité obligatoire	AccessControl, CEI, ReentrancyGuard, pause et multisignature 2-sur-3.
Audit statique	Slither 0.11.6 relancé sur les contrats finaux ; résultats bruts et triage conservés.
Bonus	Export RPC de l'historique ContributionReceived, puis heuristiques Python.
Livrables	Arborescence attendue, rapport de 10 pages, preuves locales et scénario de soutenance.

La recette de signature MetaMask, les noms des membres de l'équipe et la soutenance sont des éléments à compléter ou à démontrer par l'équipe. Aucun nom ni résultat de soutenance n'est inventé. Les captures du rapport sont celles de l'instance locale corrigée.

Architecture et choix techniques

Hardhat est retenu pour compiler, déployer et tester les contrats dans le même environnement JavaScript/TypeScript que l'interface. Ses comptes locaux et ses outils de simulation du temps permettent de vérifier les échéances, les snapshots et les attaques sans dépendre d'un réseau public. Solidity 0.8.28 compile avec optimisation à 200 passages et cible EVM Cancun.

Composant	Responsabilité et échanges
Utilisateur / MetaMask	Signe les créations, contributions, propositions, votes et retraits.
CrowdfundingCore	Détient les ETH ; mémorise les contributions ; appelle mint, burn et déblocage des CFD.
CFDToken	Solde ERC-20, verrou de transfert par détenteur, checkpoints et auto-délégation à la réception.
CampaignGovernance	Fige les poids historiques ; compte les votes ; appelle le Core après approbation.
MultiSigTreasury	Détient une trésorerie distincte ; exige une ou deux confirmations selon l'opération.
Exécuteur local	Observe les propositions exécutables et soumet execute() ; ne décide pas du résultat.
IPFS / interface	Le CID on-chain permet de lire le titre et la description ; les données externes sont validées.

Permissions au déploiement

Le déployeur reçoit les rôles administratifs et de pause. Le Core reçoit MINTER_ROLE et BURNER_ROLE sur le jeton ; seul le contrat de gouvernance reçoit GOVERNANCE_ROLE sur le Core. La trésorerie possède exactement trois adresses distinctes. Le manifeste sauvegarde le réseau, les adresses, le bloc de déploiement et les paramètres.

La trésorerie n'est pas le séquestre des campagnes. Une campagne peut la désigner comme bénéficiaire ; sinon le Core paie directement son bénéficiaire après vote. Cette distinction évite de prétendre que la règle multisignature s'applique à un contrat qui ne la met pas en œuvre.

La valeur de 100 CFD par ETH fournit une unité de lecture simple. Le taux immuable conserve la proportionnalité et empêche une modification arbitraire après contribution. Le nombre absolu ne change pas la part relative des voix.

CFD : supprimer les voix gratuites

Défaut initial

Un contributeur pouvait transférer les CFD reçus, puis récupérer son ETH après l'échec de sa campagne. Le burn était plafonné au solde restant : le destinataire conservait les jetons. Le même capital pouvait ensuite financer une autre campagne et créer de nouvelles voix. Un snapshot ne corrige pas ce défaut si l'accumulation précède la proposition.

Règle corrigée

Chaque mint de contribution augmente lockedBalance du détenteur. Les fonctions transfer et transferFrom ne peuvent déplacer que le solde non verrouillé. Le vote reste possible : le verrou protège la restitution financière, il n'empêche pas la participation politique.

Lors du remboursement, le Core détruit exactement montantWei × 100 unités CFD. Le verrou est diminué du même montant et le transfert ETH suit la mise à jour de l'état. Il n'existe plus de destruction partielle laissant en circulation des CFD issus de cette contribution remboursée.

Invariant	Conséquence
lockedBalance ≤ balanceOf	Les jetons nécessaires au remboursement restent détenus.
transfer ≤ balanceOf - lockedBalance	Un tiers ne peut pas emporter les jetons encore remboursables.
burn = contributionWei × tokensPerEth	La restitution du capital retire toutes les voix correspondantes.
Déblocage seulement après PaidOut	Les CFD deviennent transférables lorsque la campagne ne peut plus rembourser.
Déblocage une seule fois par campagne et contributeur	Une campagne payée ne peut pas déverrouiller les CFD d'une autre campagne.

Après paiement, releaseContributionTokens(id, contributeur) libère les jetons de cette contribution. Tout appelant peut déclencher la libération, mais les jetons restent chez le contributeur. Un bouton permet de le faire dans l'interface. Cette opération individuelle évite une boucle coûteuse sur tous les donateurs au moment du paiement.

Ce choix introduit une restriction de transfert explicite sur l'ERC-20 tant que la campagne est remboursable. Elle est nécessaire pour combiner remboursement garanti et conservation exacte des voix. Les restrictions doivent être expliquées aux utilisateurs.

Campagnes et autorisation de paiement

État	Opérations autorisées
En cours	Contribuer avant échéance ; soumettre un vote de déblocage anticipé ou de gel.
Réussie, sans approbation	Objectif atteint : collecte immédiatement fermée ; retrait bloqué jusqu'à la validation DAO.
Réussie, approuvée	Approbation DAO exécutée ; seul le bénéficiaire peut retirer. Les contributions sont closes.
Échouée	Objectif non atteint ; remboursement individuel et burn intégral des CFD associés.
Gelée / Annulée	Nouvelles contributions et paiement bloqués ; remboursement individuel possible hors pause.
Payée	Aucun second paiement ; les contributeurs peuvent rendre leurs CFD transférables.

Retrait normal

La collecte se ferme dès que l'objectif est atteint, sans attendre l'échéance, et toute contribution dépassant le montant restant est refusée. La durée de campagne reste l'échéance maximale : si l'objectif n'est pas atteint à cette date, les contributeurs peuvent se rembourser. L'objectif atteint ne suffit pas à payer le bénéficiaire : un détenteur de voix crée une proposition de validation du retrait final (action 2). Le vote doit satisfaire quorum, majorité et délai. L'exécution appelle approveRelease. Le bénéficiaire peut ensuite appeler withdrawFunds. Le Core marque PaidOut avant l'envoi ETH et refuse toute répétition.

Déblocage anticipé

L'action 0 permet de payer avant l'échéance, même sous l'objectif, après décision collective. L'approbation ferme la collecte et rend le paiement disponible. Elle ne peut pas être appliquée après l'échéance à une campagne échouée. Cette vérification évite qu'un vote de déblocage tardif contourne les droits de remboursement.

Une campagne approuvée anticipativement s'affiche comme réussie au sens de sa décision de financement, même si son objectif initial n'a pas été atteint. Le montant collecté et l'objectif restent visibles. Un gel ultérieur bloque le paiement tant que le Core n'a pas déjà payé.

Pause d'urgence

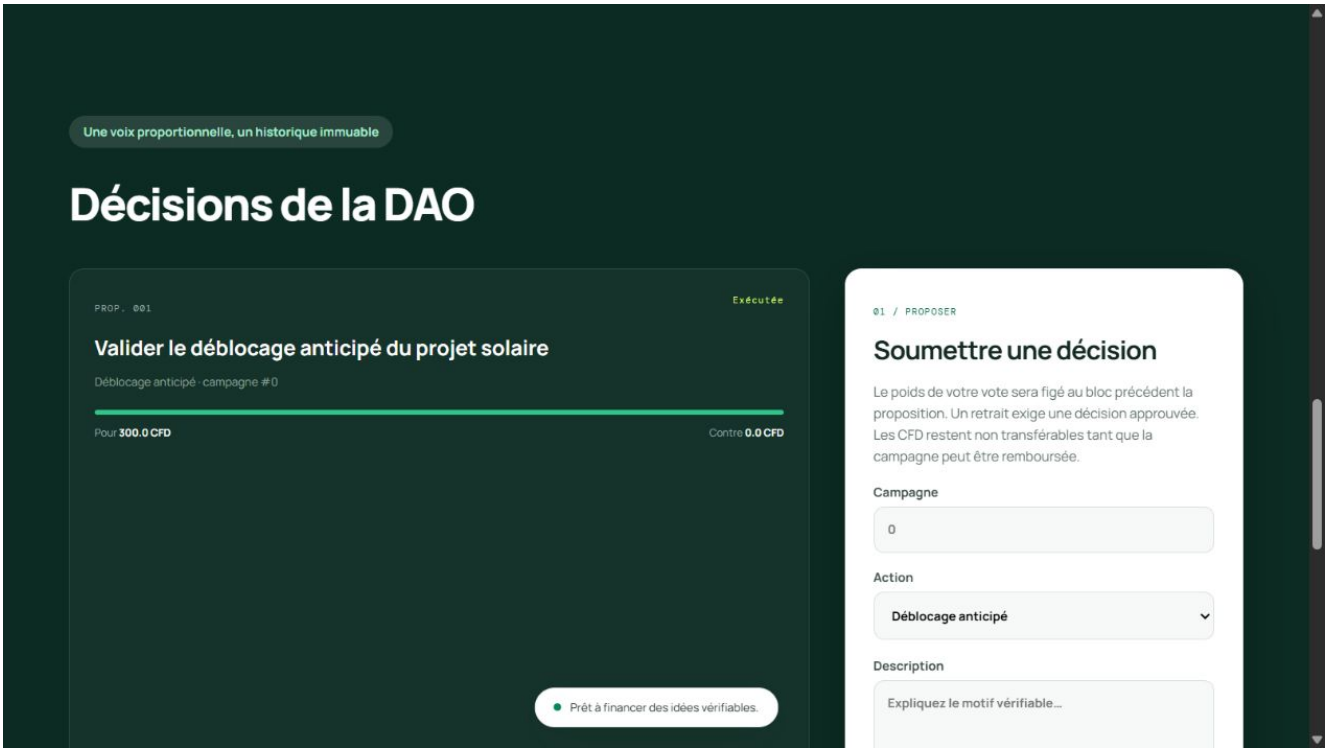
La pause du Core suspend création, contribution, retrait et remboursement. La pause du jeton bloque mint et transferts ; le burn autorisé de remboursement reste permis si le Core est actif. Les deux pauses ont ainsi des effets documentés et distincts.

Gouvernance et exécution locale

Le poids est celui du bloc précédant la proposition, calculé par getPastVotes. Il vaut exactement le solde de CFD : le jeton auto-délègue chaque adresse à la réception et désactive delegate et delegateBySig, afin qu'aucune voix ne circule indépendamment des jetons, conformément au module M3. Le quorum exige 50 % de participation et l'adoption exige strictement plus de 50 % de toute l'offre CFD historique en votes « Pour ». Les abstentions ne deviennent pas des votes contre, mais elles empêchent une minorité active de décider seule. Avec cinq détenteurs de poids égal, trois votes favorables sont nécessaires. Ce choix satisfait l'exigence du cahier des charges de définir et justifier quorum et majorité.

Le déploiement local laisse un vote ouvert 150 blocs, soit environ cinq minutes avec un bloc toutes les deux secondes. Cette fenêtre permet de changer de compte MetaMask et de faire voter plusieurs contributeurs. Après la fin du vote, un délai de 5 blocs supplémentaires précède l'exécution. La formule ajoute un bloc de séparation pour interdire une exécution au dernier bloc où le vote reste ouvert, même avec un délai configuré à zéro. La fenêtre se comptant en blocs et non en secondes, elle est réglée par réseau : Sepolia produisant un bloc toutes les douze secondes environ, le déploiement public emploie 25 blocs de vote et 2 blocs de délai pour retrouver les mêmes ordres de grandeur en temps réel. Le quorum et la majorité restent identiques sur les deux réseaux.

Une blockchain n'exécute pas spontanément un contrat lorsque le temps passe. scripts/keeper.mjs observe l'état exécutable et envoie la transaction. Sur Hardhat local, il active un bloc toutes les deux secondes et contrôle les décisions toutes les trois secondes. Il refuse les réseaux publics et utilise seulement un compte local déjà déverrouillé. Le bouton Exécuter reste disponible en secours pour un wallet connecté.



Capture réelle : proposition #1 exécutée sur le déploiement local corrigé ; 300 CFD pour, 0 contre.

L'interface relit les données toutes les quatre secondes. L'automatisation locale dépend du processus keeper en fonctionnement. Aucun service d'exécution Sepolia n'est annoncé comme déployé.

Analyse de sécurité

Menace	Défense	Risque restant
Réentrance	ReentrancyGuard et état mis à jour avant appel ETH ; test de contrat hostile.	Revue requise pour tout nouveau chemin externe.
Retrait non autorisé	onlyRole pour approuver ; bénéficiaire vérifié ; approbation obligatoire.	L'administrateur peut réattribuer des rôles.
Voix gratuites après remboursement	Verrou des CFD remboursables et burn exact.	Les voix historiques d'un vote déjà ouvert restent figées selon la règle snapshot.
Double vote par transfert	Poids historique commun et hasVoted.	Un capital durablement détenu peut dominer un vote.
Trésorerie compromise	Deux confirmations au-delà de 1 ETH et pour tout appel portant des données.	Plusieurs petits retraits restent possibles sous le seuil choisi.
Incident actif	Pause du Core par rôle dédié.	Une pause peut retarder les remboursements ; procédure de reprise nécessaire.
Perte de disponibilité IPFS	CID vérifié et passerelle configurable.	Un CID ne garantit pas un épinglage durable.
Mauvais réseau du wallet	Comparaison de chainId avant signature et réseau configuré au déploiement.	Vérification manuelle des détails affichés par le wallet toujours utile.

La trésorerie suit elle aussi Checks-Effects-Interactions : la transaction est marquée exécutée avant son appel externe. Les propriétaires sont distincts et non nuls ; la soumission vaut première confirmation. Une confirmation peut être révoquée avant exécution.

Un appel avec données exige deux propriétaires même si sa valeur ETH est nulle, car il peut transférer des ERC-20 détenus par la trésorerie. Le test reproduit ce cas. Le seuil de 1 ETH est une limite pédagogique immuable, facile à démontrer avec les comptes Hardhat ; ce n'est pas une limite économique universelle.

L'administration initiale reste une hypothèse de confiance. Avant un usage réel, transférer les privilèges vers une autorité collective et revoir les procédures de pause, d'exécution et de récupération. Ces mesures opérationnelles ne sont pas présentées comme déjà réalisées.

Tests et audit des contrats finaux

Résultat exécuté : 39 tests réussis, sans échec. npm test inclut le fichier principal et les régressions complémentaires. Les simulations se déroulent sur un réseau isolé ; elles ne modifient pas les campagnes de la démonstration.

Famille	Preuves vérifiées
Collecte et jeton	ETH séquestré, 100 CFD par ETH, montants nuls rejetés, échéance respectée.
Remboursement	Restitution et burn exact, transfert/transferFrom verrouillés, trois cycles sans accumulation.
Paiement	Vote obligatoire même après succès, bénéficiaire exclusif, paiement unique.
Déblocage CFD	Seulement après paiement, une seule fois, isolation entre campagnes.
Gouvernance	Quorum, majorité, double vote, snapshot, campagne inexistante et délai nul.
Sécurité	Administration refusée aux tiers, pause/reprise, attaque de réentrance bloquée.
Trésorerie	Seuil exact, deux signatures, données sans ETH, révocation, doublons.
Exécuteur	Décision exécutée automatiquement ; réseau public refusé.
Fuzzing reproductible	18 contributions pseudo-aléatoires ; sommes de contributions et soldes cohérents.

Analyse statique

Slither 0.11.6 a analysé 43 contrats, dépendances comprises, avec 102 détecteurs. Résultat brut : 8 alertes, aucune High. La comparaison du poids de vote à zéro produit 1 alerte Medium incorrect-equality : il s’agit ici d’un test d’éligibilité intentionnel. Quatre alertes Low timestamp concernent les échéances. Trois alertes Informational low-level-calls concernent les transferts ETH et l’appel générique multisig, avec résultat vérifié et protection de réentrance.

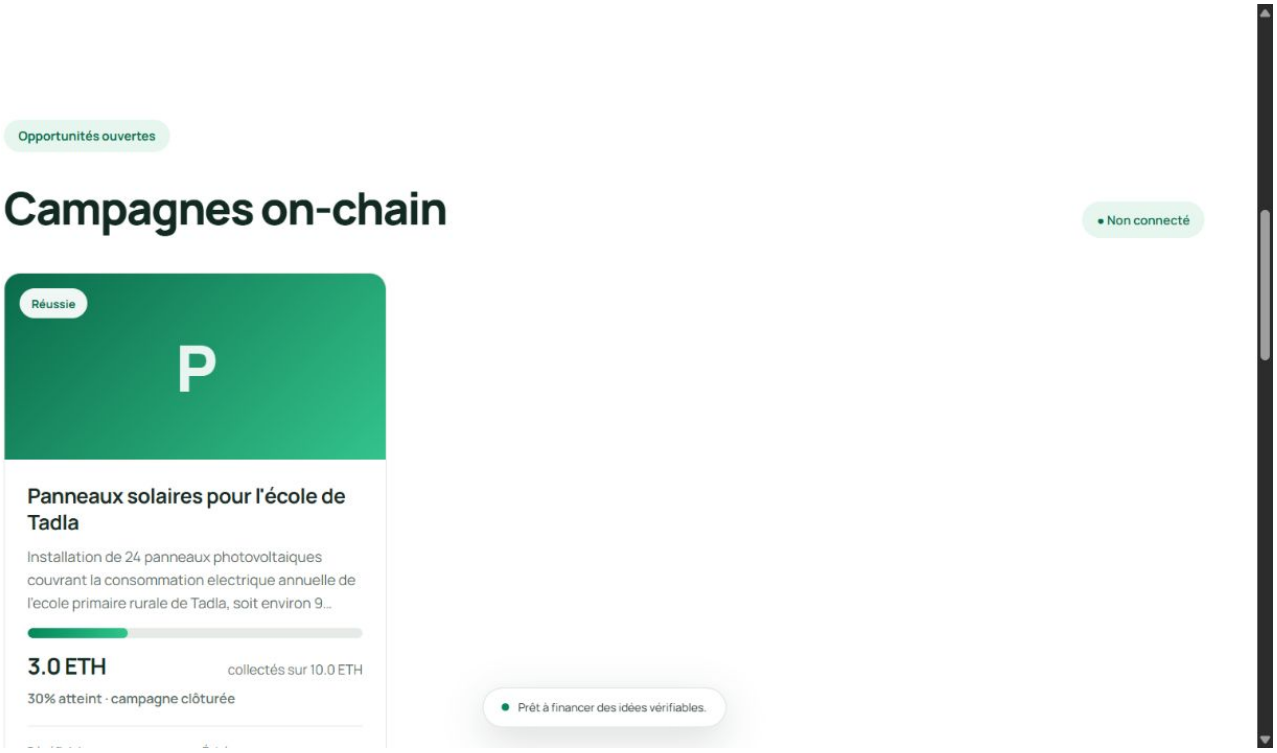
Les findings sont triés dans audit/slither-report.md et conservés dans audit/slither-final-2026-11-04.json. Une sortie Slither avec findings renvoie le code 1 ; le JSON confirme que l’analyse a réussi. L’absence d’alerte High ne prouve pas l’absence de faille métier.

Limites de validation : le fuzzing n’est pas exhaustif, le parcours de signature dans l’extension MetaMask reste à exécuter, et aucun audit indépendant n’a été réalisé. La compilation TypeScript et le build Vite ont réussi ; un avertissement de taille du bundle reste une optimisation de performance.

Déploiement corrigé et preuve locale

Réseau : Hardhat local ; chainId 31337. Le manifeste indique un premier déploiement du Core au bloc 6974. Le script génère automatiquement frontend/.env avec les adresses et le réseau cible. Les ABI utilisées par la dApp proviennent des artefacts compilés, via scripts/export-abis.mjs.

Contrat	Adresse locale
CFDToken	0x809d550fca64d94Bd9F66E60752A544199cfAC3D
CrowdfundingCore	0x4c5859f0F772848b2D91F1D83E2Fe57935348029
CampaignGovernance	0x1291Be112d480055DaFd8a610b7d1e203891C274
MultiSigTreasury	0x5f3f1dBD7B74C6B46e8c44f98792A1dAf8d69154



Capture réelle : métadonnées du CID fourni, campagne solaire et financement de démonstration de 3 ETH. Le réseau local utilise exclusivement des fonds de test.

CID : bafkreih33xpwo4wnh7u6c74zvcltj3h3giygigxlmkheqmc4mtmprxmtwy

Démarrage reproductible : npm install ; npm run check ; npm run node ; dans un second terminal, npm run deploy:local puis npm run demo:local ; lancer aussi npm run keeper:local et npm run frontend:dev. Après redémarrage complet du nœud, redéployer les contrats.

Sepolia est configurable par variables de processus : SEPOLIA_RPC_URL, TREASURY_OWNERS, FRONTEND_RPC_URL et les durées de vote. La clé de déploiement est lue dans un keystore chiffré par mot de passe (plugin hardhat-keystore), ou à défaut dans SEPOLIA_PRIVATE_KEY : le keystore évite de faire transiter la clé par l'environnement et l'historique du shell. FRONTEND_RPC_URL désigne un RPC public afin qu'aucune clé d'API ne soit incluse dans le bundle livré au navigateur. Le script vérifie la chaîne et trois propriétaires distincts. Le RPC et le chainId du front suivent le déploiement. Le déploiement Sepolia a été effectué depuis 0x568e84bB2690Dd0c4cd9b33465412C70D63901E7 : CFDToken 0x95FDb2b1f16cCF4648FE29647bB1eC4BCd025003, CrowdfundingCore 0x3371A6C82173d21d8D214cBe5045227Ec048fC15, CampaignGovernance

0x64964d2bc8419F19D20421CBcFB473494b1b03ed, MultiSigTreasury 0x4652D5c4adaEcC7bc13d021cdD45e7c604Ed7D56.La fenêtre de vote y vaut 25 blocs et le délai d’exécution 2 blocs, soit environ cinq minutes et une demi-minute au rythme deSepolia. Les trente contrôles on-chain de npm run verify:sepolia passent sur ces adresses.

Recette, bonus et limites restantes

Recette de démonstration en dix minutes

Temps	Démonstration
0-2 min	Présenter les quatre contrats, les rôles et le séquestre des fonds.
2-4 min	Créer une campagne avec le CID fourni, contribuer et montrer les CFD et le pouvoir de vote immédiat.
4-6 min	Créer un vote, montrer le snapshot, voter et observer l'exécution après délai.
6-8 min	Montrer le paiement autorisé, le refus sans vote et le remboursement avec burn exact dans les tests.
8-10 min	Démontrer la réentrance bloquée, la pause et le seuil multisig ; commenter Slither.

Le parcours réel MetaMask doit confirmer : connexion et changement de réseau, deux comptes de contribution, refus de signature sans effacement du formulaire, vote, retrait par le bénéficiaire et remboursement. L'interface ne remplace pas les contrôles on-chain. Aucun écran administratif ou de trésorerie n'est ajouté : ces opérations restent démontrables par scripts et tests, conformément au périmètre minimal de M4.

Bonus hors chaîne

npm run analyze:local exporte les événements ContributionReceived depuis le bloc de déploiement, récupère leurs horodatages et lance le script Python. La chaîne est vérifiée par rapport au manifeste. Les heuristiques signalent au moins quatre montants identiques, une adresse représentant au moins 60 % du volume et une heure au moins quatre fois supérieure à la médiane. Un export réel de la campagne locale a été analysé. La concentration d'une adresse unique est un signal attendu, pas une preuve de fraude.

Limites et suite

Le keeper fourni automatise seulement Hardhat local. Un réseau public exige son propre service opérateur. Les contrats ne sont pas évolutifs et une correction exige un nouveau déploiement. Les snapshots n'empêchent pas la domination par un gros détenteur. Le pinning IPFS doit être maintenu. Le dépôt doit être relu et enregistré dans Git avant livraison. Le rapport antérieur de 39 pages est remplacé comme référence technique par ce document de 10 pages, sans inventer d'identité d'équipe. Le déploiement Sepolia est réel et vérifiable aux adresses indiquées ; la démonstration en direct reste conduite sur Hardhat local, où le keeper exécute les décisions automatiquement.

Sources de référence

Cahier des charges mini_projet_blockchain.pdf, pages 1-2 ; code source des quatre contrats ; tests CrowdfundDAO.ts et verification.test.ts ; sortie Slither finale ; manifeste localhost.json ; captures réelles sous docs/screenshots. Les références externes de bibliothèque sont les paquets installés OpenZeppelin, Hardhat et Ethers.js. Les résultats présentés correspondent à la version locale du 4 novembre 2026.